LAB 5: STATIC CODE ANALYSIS

Bank Account System

Tools: Pylint, Bandit and Flake8

Name:Shreenandan jamadar

SRN:PES2UG24CS479

Section :H

1. Objective

To identify and fix code quality, security, and formatting problems in a Python bank account system using Pylint, Bandit, and Flake8.

2. Program Used

The program demonstrates a bank account system with account-related data handling, bonus calculation, PIN hashing, temporary PIN generation, and data backup functionality. Static analysis tools were used to identify and resolve potential security vulnerabilities and code quality issues.

Tool	Purpose
Pylint	Checks Python code quality and coding practices.
Bandit	Checks common security problems in Python code.
Flake8	Checks code style and formatting.

3. Problems Found and Fixed

Problem	Tool	Fix
Unsafe use of `eval()`	Security review	Replaced unsafe evaluation with safer input handling, where applicable.
Weak MD5 hashing	Security review	Replaced MD5 with SHA-256 for the PIN-hashing demonstration.
Insecure random-number generation	Bandit	Used `secrets.randbelow()` for temporary PIN generation.
Unsafe shell execution	Bandit	Removed or corrected unsafe shell-command execution.
Broad exception handling	Pylint	Used more specific exception handling.
File handling improvements	Code review	Used a context manager and explicit encoding for backup operations.
Naming and formatting issues	Pylint / Flake8	Improved naming consistency and code formatting.

4. Pylint Results

Before: 5.62/10

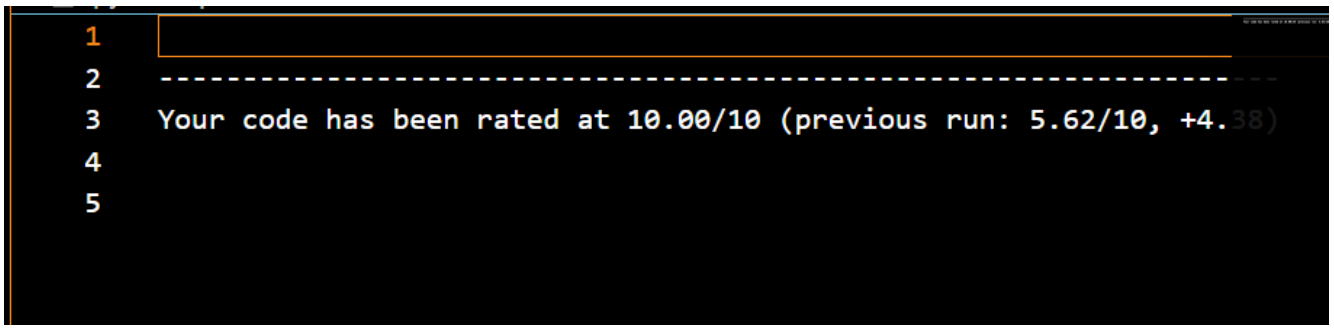
After: 10.00/10

The final code achieved a Pylint score of 10.00/10, indicating an improvement in the code quality checks reported by Pylint.

Pylint – Before

```
1  ************* Module bank_account_system
2  bank_account_system.py:70:0: C0304: Final newline missing (missing-final-newline)
3  bank_account_system.py:11:0: C0116: Missing function or method docstring (missing-function-do
4  bank_account_system.py:11:0: C0103: Function name "createAccount" doesn't conform to snake_ca
5  bank_account_system.py:11:0: W0102: Dangerous default value [] as argument (dangerous-default
6  bank_account_system.py:15:24: C0209: Formatting a regular string which could be an f-string (
7  bank_account_system.py:19:0: C0116: Missing function or method docstring (missing-function-do
8  bank_account_system.py:22:4: W0702: No exception type(s) specified (bare-except)
9  bank_account_system.py:26:0: C0116: Missing function or method docstring (missing-function-do
10 bank_account_system.py:28:7: C0121: Comparison 'account_data.get(acc_id) == None' should be '
11 bank_account_system.py:30:4: R1705: Unnecessary "else" after "return", remove the "else" and
12 bank_account_system.py:27:4: W0612: Unused variable 'service_fee' (unused-variable)
13 bank_account_system.py:37:0: C0116: Missing function or method docstring (missing-function-do
14 bank_account_system.py:39:12: W0123: Use of eval (eval-used)
15 bank_account_system.py:38:4: W0612: Unused variable 'bal' (unused-variable)
16 bank_account_system.py:43:0: C0116: Missing function or method docstring (missing-function-do
17 bank_account_system.py:47:0: C0116: Missing function or method docstring (missing-function-do
18 bank_account_system.py:51:0: C0116: Missing function or method docstring (missing-function-do
19 bank_account_system.py:52:8: W1514: Using open without explicitly specifying an encoding (uns
20 bank_account_system.py:52:8: R1732: Consider using 'with' for resource-allocating operations
21 bank_account_system.py:58:0: C0116: Missing function or method docstring (missing-function-do
22 bank_account_system.py:6:0: W0611: Unused import sys (unused-import)
23
24 -----------------------------------
25 Your code has been rated at 5.62/10
26
27
```

Pylint – After



5. Bandit Results

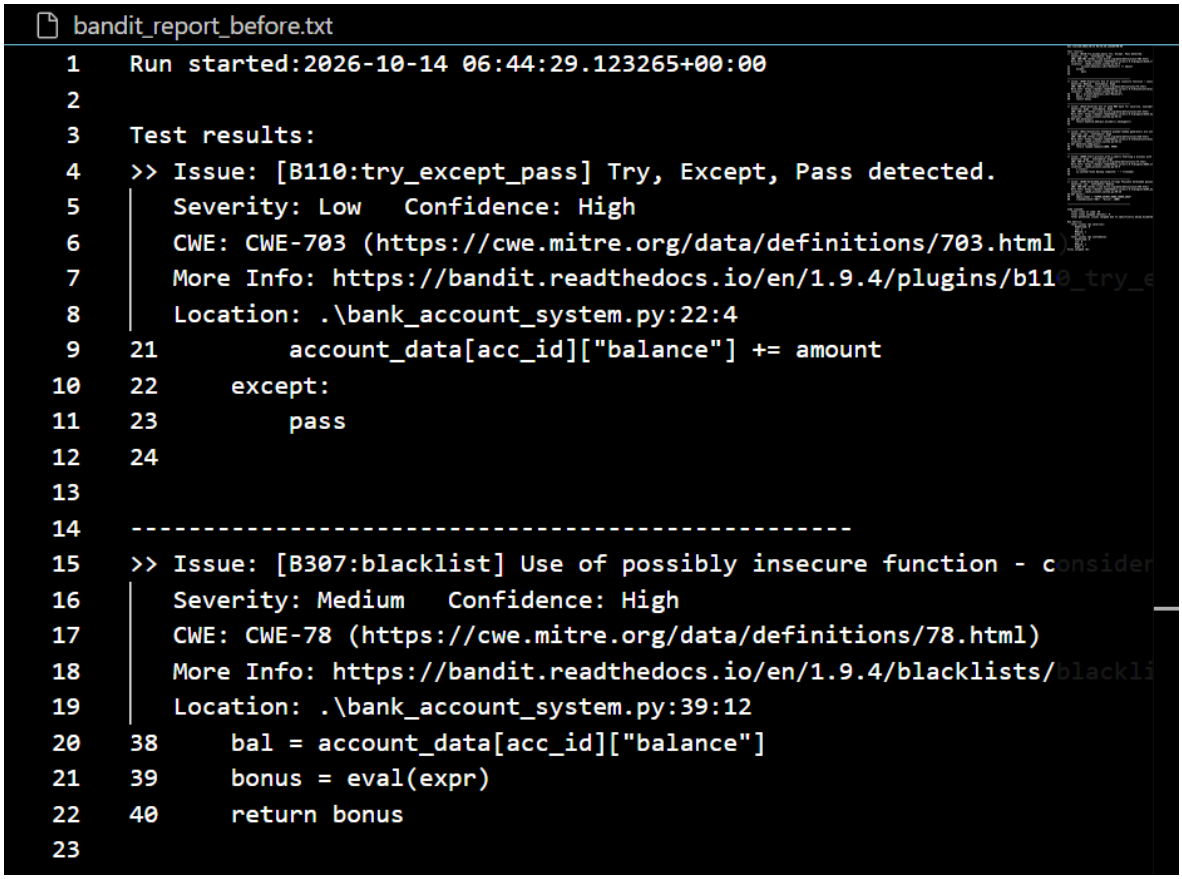
Before: Security-related findings were reported.

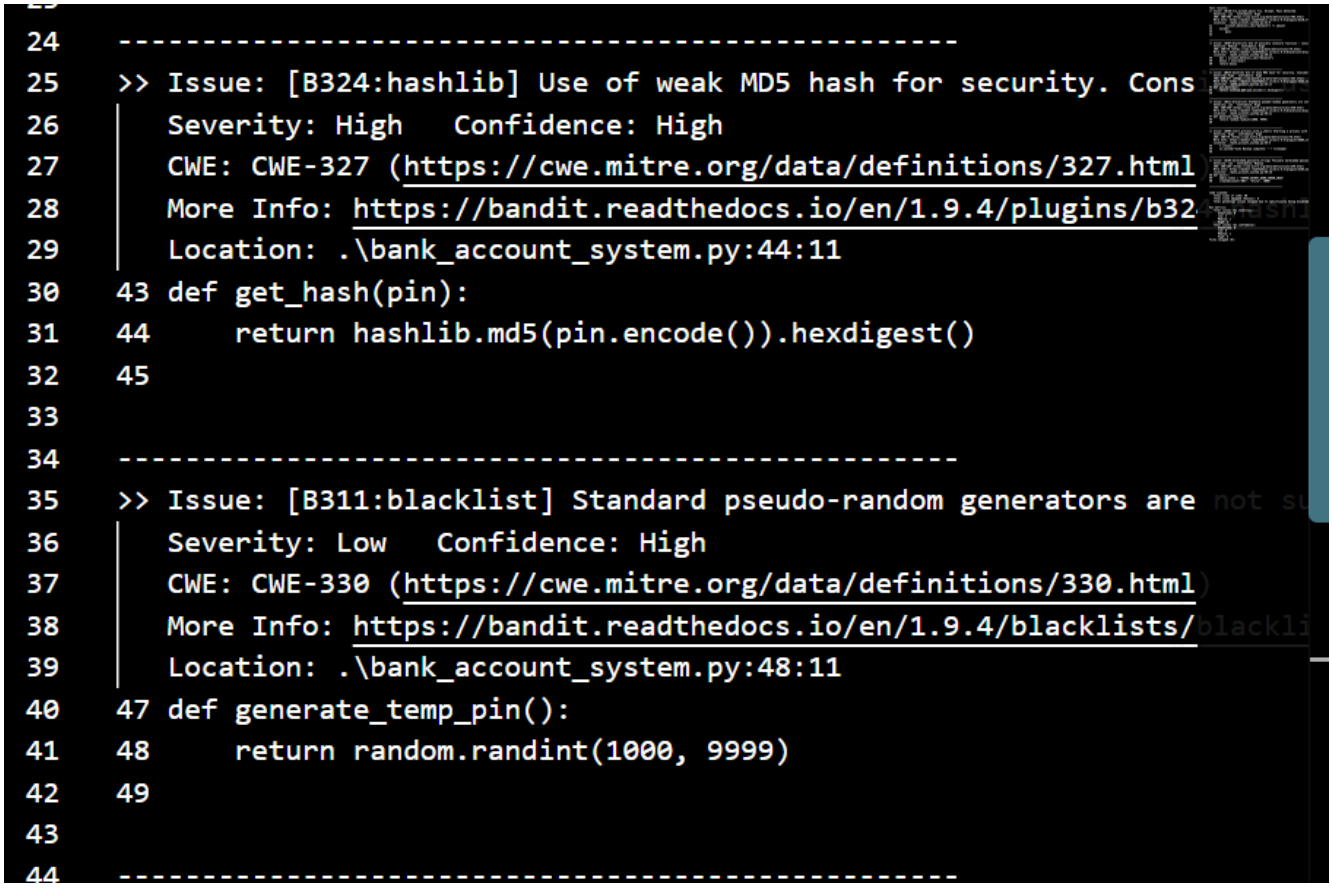
After: No issues identified.

The final Bandit report showed no issues identified by the too

.

Bandit – Before





>> Issue: [B324:hashlib] Use of weak MD5 hash for security. Cons
Severity: High Confidence: High
CWE: CWE-327 (https://cwe.mitre.org/data/definitions/327.html
More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b32
Location: .\bank_account_system.py:44:11
43 def get_hash(pin):
44 return hashlib.md5(pin.encode()).hexdigest()
45

>> Issue: [B311:blacklist] Standard pseudo-random generators are
Severity: Low Confidence: High
CWE: CWE-330 (https://cwe.mitre.org/data/definitions/330.html
More Info: https://bandit.readthedocs.io/en/1.9.4/blacklists/
Location: .\bank_account_system.py:48:11
47 def generate_temp_pin():
48 return random.randint(1000, 9999)
49
----

bandit_report_before.txt

```
>> Issue: [B605:start_process_with_a_shell] Starting a process with a shell, possible injection detecte
   Severity: High   Confidence: High
   CWE: CWE-78 (https://cwe.mitre.org/data/definitions/78.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b605_start_process_with_a_shell.html
   Location: .\bank_account_system.py:55:4
54     f.close()
55     os.system("echo Backup complete: " + filename)
56
--------------------------------------------------
>> Issue: [B105:hardcoded_password_string] Possible hardcoded password: 'SUPER_SECRET_BANK_TOKEN_2026'
   Severity: Low   Confidence: Medium
   CWE: CWE-259 (https://cwe.mitre.org/data/definitions/259.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b105_hardcoded_password_string.html
   Location: .\bank_account_system.py:59:18
58 def main():
59     admin_token = "SUPER_SECRET_BANK_TOKEN_2026"
60     createAccount("A01", "Alice", 1000)
63
--------------------------------------------------

Code scanned:
   Total lines of code: 50
   Total lines skipped (#nosec): 0
   Total potential issues skipped due to specifically being disabled (e.g., #nosec BXXX): 0

Run metrics:
   Total issues (by severity):
      Undefined: 0
      Low: 3
      Medium: 1
      High: 2
   Total issues (by confidence):
      Undefined: 0
      Low: 0
      Medium: 1
      High: 5
Files skipped (0):
```

Bandit – After

bandit_report_after.txt

```
Run started:2026-10-14 09:35:06.745467+00:00

Test results:
   No issues identified.

Code scanned:
   Total lines of code: 69
   Total lines skipped (#nosec): 0
   Total potential issues skipped due to specifically being disabled (e.g., #nosec BXXX): 0

Run metrics:
   Total issues (by severity):
      Undefined: 0
      Low: 0
      Medium: 0
      High: 0
   Total issues (by confidence):
      Undefined: 0
      Low: 0
      Medium: 0
      High: 0
Files skipped (0):
```

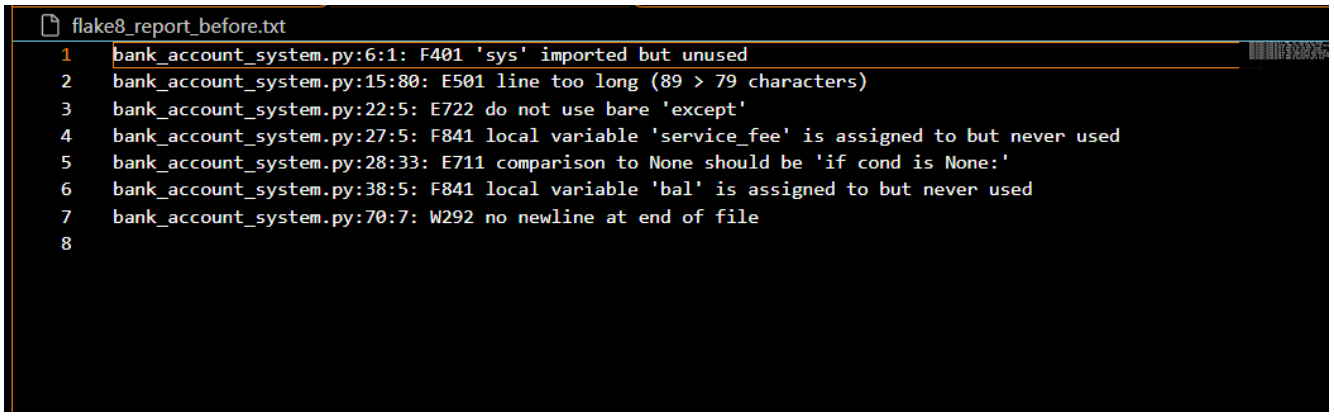
6. Flake8 Results

Before: Formatting and style issues were reported.

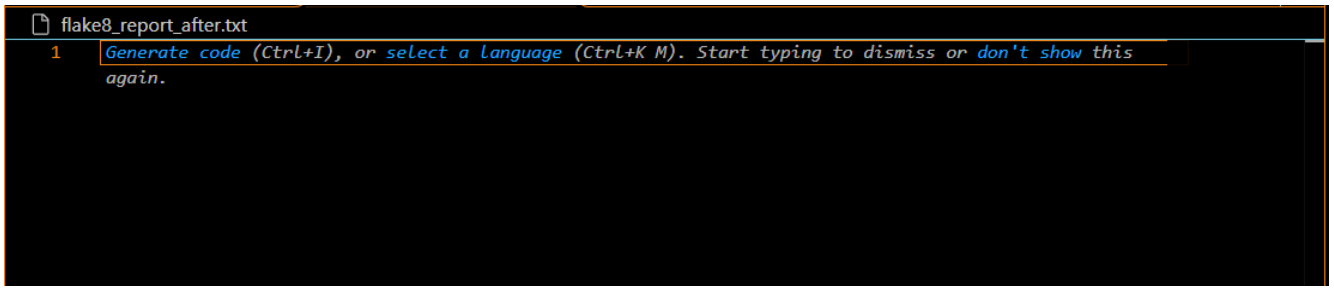
After: No output was produced in the after-report.

The after-report was empty. Confirm that Flake8 executed successfully before describing this as a clean result.

Flake8 – Before



Flake8 – After



7. Program Verification

After fixing the identified issues, the bank account system was executed again to verify its behavior. The program displayed the bonus calculation, generated a PIN hash, generated a temporary PIN, and displayed a message when the admin token was not configured.

The observed output was:

- Bonus: 65.0

- PIN Hash: Generated using SHA-256
- Temporary PIN: Generated securely using the `secrets` module
- Admin token: Not configured

The temporary PIN changes between executions because it is generated dynamically.

8. Reflection

What did I learn?

I learned how static code analysis tools can identify potential security vulnerabilities, coding errors, and formatting issues without executing the code. I also learned how to improve Python code using the findings from Pylint, Bandit, and Flake8.

Which tool was most useful?

Pylint was useful for identifying code quality and naming issues. Bandit helped identify potential security vulnerabilities, while Flake8 helped check formatting and style consistency.

How did the tools improve the code?

The tools helped identify problems that needed correction. After making the changes, the Pylint score improved from 5.62/10 to 10.00/10, and the Bandit after-report showed no issues identified.

How can these tools be used in a project?

These tools can be integrated into the development workflow to check code quality, identify security risks, and maintain consistent coding standards. They can also be incorporated into automated checks during development and continuous integration.

9. Conclusion

In this lab, the Bank Account System was analyzed using Pylint, Bandit, and Flake8. The identified code quality, security, and formatting issues were reviewed and addressed. The Pylint score improved from 5.62/10 to 10.00/10, and the final Bandit report showed no issues identified. The Flake8 after-report was empty, so successful execution of the analysis command should be confirmed. This lab demonstrated the importance of static code analysis in improving code quality, security, and maintainability.